



university of
 groningen

Potential-Based Reward Shaping for Planar Pushing

Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026



Overview – Thesis Structure

1. The Problem

- MDP Formulation
- Push Mechanics (Limit Surface)
- Old Ad-hoc Reward Failure

2. PBRs Solution

- PBRs Theory
- Potential Functions
- SE(2) Unified Distance Metric

3. What We Built

- **A:** PBRs Single-Agent
- **B:** Forced Curriculum
- **C-D:** ASP (Alice/Bob)
- **E-H:** ASP Variants (SE(2), disc, time-based)

4. What We Found

- PBRs outperforms ad-hoc reward
- ASP severely underperforms
- Reward and curriculum are independent
- Simplest model wins every time

Research Questions



Three Research Questions

1. **RQ1:** Can potential-based reward shaping replace hand-tuned dense rewards for planar pushing tasks?
2. **RQ2:** Does curriculum learning or asymmetric self-play improve sample efficiency over a direct single-agent approach?
3. **RQ3:** Are reward design and curriculum design *independent* problems in contact-rich manipulation?

MDP Formulation – State & Action

State $s \in \mathbb{R}^{28}$:

$$s = [\underbrace{\mathbf{q}_{ee}}_{\text{EE pose(6)}} \mid \underbrace{\mathbf{p}_{obj}, \mathbf{R}_{obj}}_{\text{obj state(14)}} \mid \underbrace{\mathbf{p}_{goal}, \mathbf{R}_{goal}}_{\text{goal pose(6)}} \mid \underbrace{d_{pos}, d_{rot}}_{\text{errors(2)}}]$$

Action $\mathbf{a} \in \mathbb{R}^4$ (object-relative):

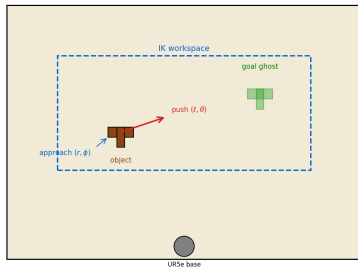
(r, ϕ, ℓ, θ) with 21 bins/dim $\Rightarrow 194\,481$ discrete actions

$$X_s = x_{obj} + r \cos(\psi_{obj} + \phi)$$

$$Y_s = y_{obj} + r \sin(\psi_{obj} + \phi)$$

$$X_f = X_s + \ell \cos \theta, \quad Y_f = Y_s + \ell \sin \theta$$

Planar Pushing Task — Object-Relative Action (r, ϕ, ℓ, θ)





MDP Formulation – Transition & Episode

Transition – quasi-static limit surface (Mason 1986): $\mathbf{t} = (v_x, v_y, \omega) \propto \nabla f(\mathbf{w})$. Object twist is normal to the surface – every push couples translation *and* rotation.

Episode (training): max 15 pushes; **Validation:** 30 pushes, best-of-20 retries.

Success: pos < 5 cm **and** rot < 0.2 rad. Ends early on catastrophe (tipped / launched / OOB).

Table 1.40×1.00 m; IK $X \in [-0.5, 0.5]$, $Y \in [0.25, 0.70]$ m.

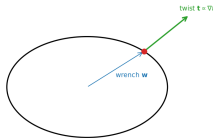
SE(2) Unified Distance – Models E-H

SE(2) metric: $d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$

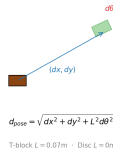
- Single scalar combining (dx, dy) and $d\theta$; L = char. length (T-block 0.07m, disc 0m)
- **Single-potential PBRs:** $\Phi(s) = w e^{-k d_{\text{pose}}^2}$, $k = 30$, $w = 10$
- Eliminates competing pos/rot gradients
 - aligns reward with the coupled physics

Result: SE(2) does *not* rescue ASP – E/F reach only 6.7% validation SR vs single-agent 80%.
The self-play curriculum is the root cause, not the metric.

Limit Surface Normality
(Goyal et al. 1991)



SE(2) Unified Distance





Old Reward Formula – Why It Failed

Fractional Improvement Formula (Fixes P63-P64)

$$R = \underbrace{\alpha \cdot \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}}}_{\text{position improvement}} + \underbrace{\alpha \cdot \frac{y_{\text{prev}} - y_{\text{now}}}{y_{\text{prev}}}}_{\text{rotation improvement}} - \underbrace{\beta_{\text{pos}} \cdot d_{\text{now}}}_{\text{position penalty}} - \underbrace{\beta_{\text{rot}} \cdot y_{\text{now}}}_{\text{rotation penalty}}$$

$$\alpha = 3.0, \beta_{\text{pos}} = 0.5, \beta_{\text{rot}} = 0.25$$

$d_{\text{prev}}, d_{\text{now}}$: position error before/after push; $y_{\text{prev}}, y_{\text{now}}$: rotation error before/after push

Problem 1 – Non-Markov: Depends on d_{prev} – same state transition gets different reward from different starting positions. Value function ill-defined.

Result: The fractional formula suffers from two structural flaws – non-Markov dependence and

Problem 2 – Gradient Starvation: 1cm improvement from 30cm:

$\alpha \cdot 0.01 / 0.30 - \beta_{\text{pos}} \cdot 0.29 = -0.045$. Progress gets **negative** reward. $\mathbb{E}[R] \approx -0.15$, near-zero variance.



PBRS – The Shaping Theorem

Ng et al. (1999):

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_{\text{shaping reward } F}$$

Policy invariance: $Q'(s, a) = Q(s, a) - \Phi(s)$ – optimal actions unchanged.

Our potential functions (Models A/B):

$$\Phi(s) = w_{\text{pos}} e^{-k_p d_{\text{pos}}^2} + w_{\text{rot}} e^{-k_r c(s)}, \quad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

$k_p=30, k_r=5, w_{\text{pos}}=w_{\text{rot}}=10.0$

Dense reward: $F(s, s') = \Phi(s') - \Phi(s)$ – difference only, no constant drain.

Episodic PBRS (Grześ 2017): $\gamma_{\text{shaping}}=1.0$ valid – $\sum F = \Phi(s_T) - \Phi(s_0)$ bounded.

PBRs – Why It's the Right Choice for Planar Pushing

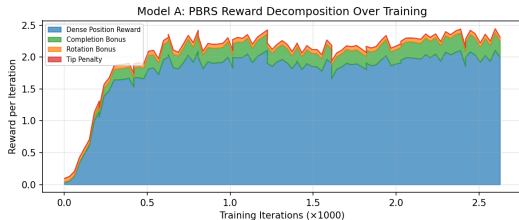
Properties of $F(s, s') = \Phi(s') - \Phi(s)$:

1. **Markovian** – depends only on s, s' , not history. Old formula violated this by dividing by d_{prev} .
2. **Bounded** – exponential potentials prevent reward explosions from PhysX contact spikes.
3. **Zero-sum cycles** – detour pushes net to zero; agent free to explore without reward hacking.
4. **No constant drain** – $F=0$ when stationary; no penalty for hesitation or deliberation.

Key consequence: you can replace any ad-hoc reward with a potential-based one *without changing what the agent learns* – only *how fast* it learns. Reward design and policy optimisation are decoupled.

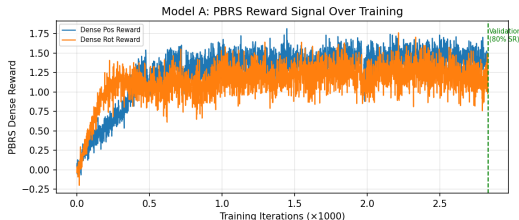


PBRS – Reward Signal Analysis



Reward diet: ~85% dense PBRS shaping,
+5/+2 sparse bonuses, penalties.

Dense signal on every push, bounded rewards, stable value – PBRS avoids the gradient starvation and critic instability of the old fractional formula.



Signal vs performance: bounded exponential
potentials → stable, correlated gradient.

Model A – PBRs Single-Agent (Our Best Model)



Architecture: Single PPO agent, LSTM(28→256) + MLP[512, 256] trunk, MultiCategorical head (4 dims $\times$ 21 bins). No curriculum, no Alice/Bob, no GoalEncoder.

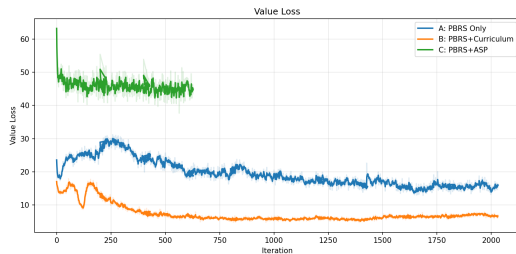
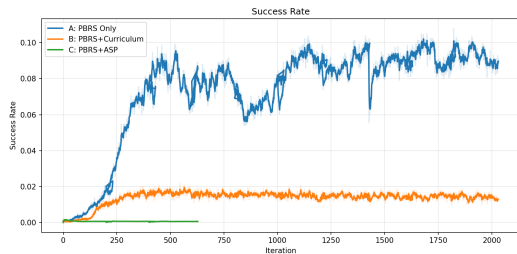
Training: 528 envs, 15 pushes/rollout, ~ 3000 iterations ~ 1.6 days wall-clock (~ 0.022 it/s).

Why it works:

1. **Dense, bounded gradient** – PBRs fires on every push
2. **Fixed goal distribution** – stationary random goals; no adversarial shift
3. **Single network** – one agent, one optimizer; no multi-agent instability



Model A – Training Dynamics

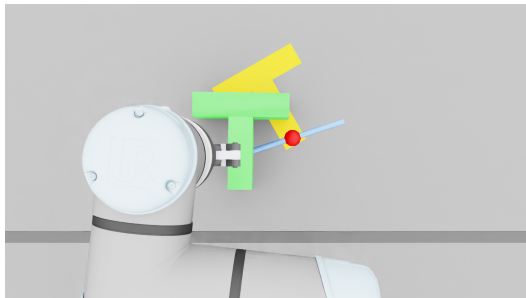


Left: Model A (blue) steadily improves training success rate over 3000 iterations. Model B (orange) follows a similar trajectory but converges slightly lower. ASP variants (E-H) remain near baseline throughout. **Right:** Stable value function – no explosion. Initial $V \approx 0.057$ with gain= 0.01.

Model A – Validation Results

30 held-out T-block scenes (best-of-20):

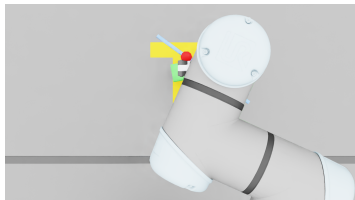
- **80.0% SR** (24/30) – pos-only **100%**, pos+rot **70%**
- PosErr **0.032 m**, RotErr **0.568 rad**
- Training SR ($\sim 9.4\%$) understates validation – strict combined gate vs best-of-20



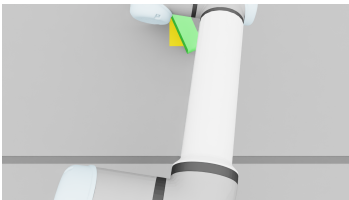
Overhead view – Model A solving a pos+rot scene ▶

[play clip](#)

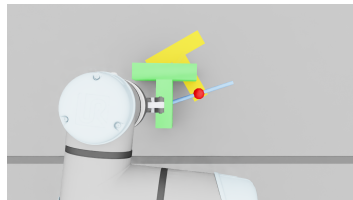
Model A – Validation Runs (overhead)



Forward (pos-only) ▶ [play clip](#)



Lateral push (pos-only) ▶ [play clip](#)

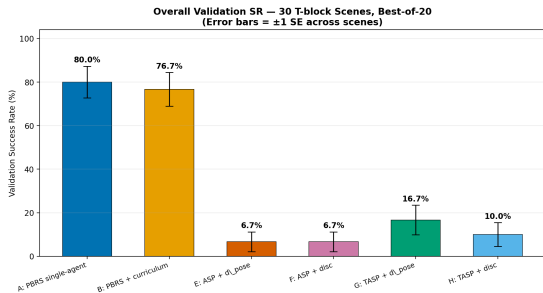


Translate + rotate ▶ [play clip](#)

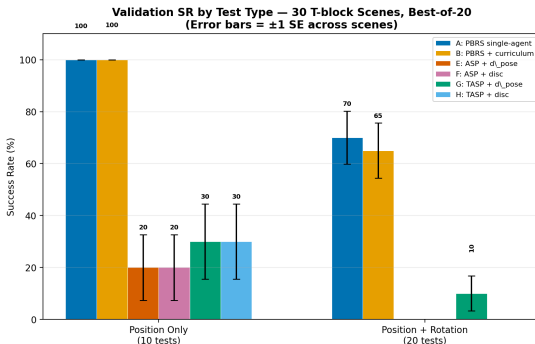
[Ng et al. 1999] [Grześ 2017] [Schulman et al. 2017]



Multi-Model Validation – with Confidence Intervals



Overall SR & SE – 30 T-block scenes, best-of-20



SR by test type: pos-only (10 tests) vs pos+rot (20 tests)

A vs B: 80.0% vs 76.7% – 3.3pp gap **not statistically significant** ($p > 0.05$ on 30 independent scenes). **ASP (E-H):** 4.8–11.9 $\times$ below single-agent – gap is significant. *Error bars = ± 1 standard error across scenes. Model C (base ASP) excluded – has no Isaac best-of-20 validation (0% SR on 20 older scenes). Model*

Model B – PBRs + Forced Curriculum



Design: Same PPO agent as Model A + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRs (multi-objective)

Original trigger was mis-specified: the first version used a per-push mean position error threshold of 0.08m – below even the *best* model's validation PosErr (0.202m). The curriculum never activated because the threshold was unreachable by construction (P82 fix).

Corrected trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below Model A.



Model B – Results: A_{simple} vs $B_{\text{curriculum}}$

Metric	Model A (no curric.)	Model B (curriculum)	Gap
Validation SR	80.0%	76.7%	1.04×
PosErr (m)	0.032	0.023	B 28% lower
RotErr (rad)	0.568	0.663	A 14% lower
Pos-only SR	100%	100%	equal
Pos+rot SR	70%	65%	5pp

Result: Curriculum improves position precision (0.023 vs 0.032 m) from the pos-only phase, but slightly degrades rotation (0.663 vs 0.568 rad). Net effect: B is 3.3pp behind A. **Statistical note:** The 3.3pp A vs B gap is **not** significant ($p > 0.05$, two-tailed z-test; SE of difference = ± 10.6 pp across 30 independent scenes). Both A and B solve the task similarly well. **Takeaway:** A staged curriculum *can* work alongside PBRs, but does not beat the simpler no-curriculum baseline.



Models C-H – ASP Architecture (Alice/Bob)

Alice (Goal Proposer):

- PPO agent, free-roaming push phase – leaves T-block in a new SE(2) pose
- That pose becomes Bob's target goal
- Reward: +5 if Bob fails to reproduce it, -1 if Bob succeeds

Bob (Goal Achiever):

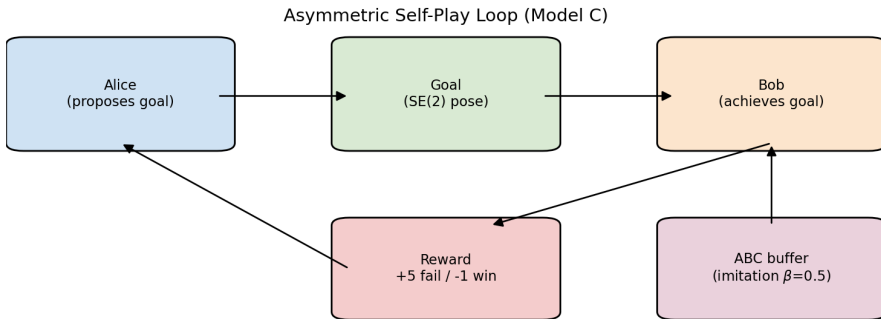
- PPOABC + GoalEncoder – must reproduce Alice's goal pose from scratch
- Reward: PBRS dense shaping + sparse completion bonuses (+5/+2)
- Failed trajectories stored in ABC buffer for behavioral cloning ($\beta_{abc}=0.5$)

Variants (E-H): SE(2) unified d_{pose} metric (T-block & disc) $\times$ {outcome-based, time-based} Alice reward. **C/D excluded:** C has no Isaac eval; D is GoalEncoder ablation (no measurable difference).



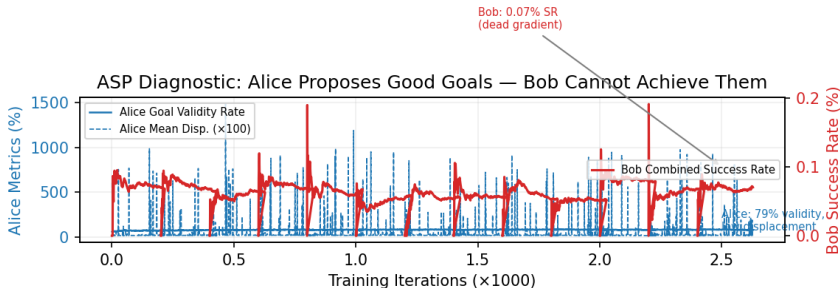
ASP Loop – Alice↔Bob Cycle

Adversarial training cycle: Alice creates a goal → Bob attempts it → outcome feeds back to Alice





ASP Diagnostic – Alice vs. Bob



Alice proposes valid, low-displacement goals, yet Bob cannot achieve the combined pos+rot objective under the non-stationary adversarial goal distribution.



Models E-H – ASP Variants

2×2 design on top of Model C: Alice reward type × reward metric.

	SE(2) d_{pose} (T-block)	SE(2) d_{pose} (disc)
Outcome-based Alice (+5/ − 1, $\beta_{\text{abc}} = 0.5$)	E – $R_A = +5$ fail/−1 win Valid SR: 6.7%	F – same, disc object Valid SR: 6.7%
Time-based Alice ($\gamma_{\text{sp}} = 0.5$, no ABC)	G – $R_A = \gamma_{\text{sp}} \cdot \max(0, t_B - t_A)$ Valid SR: 16.7%	H – same, disc object Valid SR: 10.0%

Findings:

- Time-based Alice (G/H) outperforms outcome-based (E/F): +10pp (T-block), +3.3pp (disc)
- T-block outperforms disc across both reward types
- Best ASP variant (G, 16.7%) still **4.8**× below single-agent (A, 80.0%)



Models E-H – Validation Comparison

Model	Variant	Valid SR	PosErr	RotErr
A	Single-agent (ref.)	80.0%	0.032 m	0.568 rad
G	TASP + d_pose	16.7%	0.158 m	1.457 rad
H	TASP + disc	10.0%	0.186 m	1.260 rad
E	ASP + d_pose	6.7%	0.143 m	1.612 rad
F	ASP + disc	6.7%	0.197 m	1.576 rad

Key findings:

- Time-based ASP (G/H) is $2.5\times$ better than outcome-based (E/F), but still **4.8** $\times$ **below** single-agent A
- No ASP variant achieves pos+rot SR above 10% – the combined gate remains the bottleneck
- ASP complexity (2 agents, GoalEncoder, ABC/historical pool) provides no benefit



Why ASP Fails – Failure Modes

1. **Non-stationary goal distribution:**

- Alice learns continuously → Bob's training distribution shifts every iteration
- PPO's importance ratio computed against stale rollouts from a previous Alice
- Distribution shift dominates the ratio – PPO clip fires on most transitions

2. **Confounded ablation (C6):**

- ASP changes 3 axes at once: goal distribution, agent count, episode budget
- The collapse could be due to harder/non-stationary goals, not the curriculum mechanism itself
- Time-based ASP (G, 16.7%) shows partial rescue – self-regulating curriculum helps



Why ASP Fails – The Evidence

What we observe:

- Outcome-based ASP (E/F): 6.7% – essentially random
- Time-based ASP (G): 16.7% – partial rescue, +10pp over E
- All ASP variants have RotErr > 1.2 rad – rotation **never learned**
- Even the best variant (G) is $4.8\times$ below single-agent

Why this is not just a budget issue:

- All models trained for 3000 iters at 528 envs on identical hardware
- ASP and single-agent have identical per-iteration wall-clock (~ 0.022 it/s)
- The gap is **structural**, not due to compute starvation

Combined effect: The non-stationary goal distribution + multi-axis design confounds prevent Bob from learning the combined pos+rot objective. Time-based Alice partially mitigates the distribution shift but cannot close the gap to single-agent performance.



Caveat – Confounded Ablation (C6)

Problem: ASP (E–H) differs from single-agent (A) on *at least 4 axes simultaneously*:

Variable	Single-agent (A, 80.0%)	ASP (E–H, 6.7–16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (E/F) / No (G/H)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

The $4.8\times$ gap cannot be attributed to any single lever.

- Models G/H (time-based Alice, no ABC) vs E/F (outcome-based, ABC enabled) isolate the Alice reward axis – G gains +10pp
- But this is a *single* 1-axis experiment within a 7-axis package change
- The independence claim (Slide 10) concerns **reward design vs curriculum design**, not

Key Result – Reward & Curriculum Are Independent



Model	Valid SR	vs Single-Agent
A – Single-agent PBRs	80.0%	– (baseline)
B – PBRs + Curriculum	76.7%	–3.3pp
G – PBRs + Time-based ASP	16.7%	4.8× gap
H – PBRs + TASP (disc)	10.0%	8.0× gap
E – PBRs + Outcome ASP	6.7%	11.9× gap
F – PBRs + ASP (disc)	6.7%	11.9× gap

The Independence Finding

- **Curriculum adds no value:** B_curr (76.7%) trails A_simp (80.0%) by 3.3pp, but this gap is **not statistically significant** ($p > 0.05$). Curriculum is competitive – just not better.
- **ASP destroys performance:** substituting self-play for single-agent reduces SR 4.8–11.9×, regardless of Alice reward structure or object type.

Computational Overhead – ManagerBasedRL vs DirectRL



ASP per-iteration cost (measured, 528 envs):

~**1.0x** vs single-agent – 2-agent machinery adds **no** wall-clock overhead. Shared cuRobo IK + physics dominates.

Where overhead matters: ManagerBasedRLEnv (ASP pipeline) vs DirectRLEnv (single bare PPO). Table Tennis DirectRL: **4096 envs at ~20 it/s** → 2.1M transitions/iteration. The full ManagerBasedRL API stack (Action, Observation, Event, Termination managers) imposes 5–10x throughput penalty vs direct tensor operations.

Key result:

- ASP is NOT slower per iteration – it simply **does not learn**
- ManagerBasedRL overhead is a **separate** architectural issue, not specific to ASP



Isaac Lab – the Right Venue for ASP

Why GPU-batched simulation is necessary for this comparison:

Throughput (push-macros/s, measured):

Setting	push/s	1M pushes
Isaac (1 GPU, 528 env)	172	~1.6 h
gym A/B (6 CPU)	91	~3.0 h
gym ASP (1 CPU)	22.4	~12.4 h

ASP on GPU vs CPU:

- GPU: ASP & single-agent identical throughput (~ 0.022 it/s)
- CPU: ASP forced single-process $\rightarrow \sim 7.7\times$ slower than A/B

Three pillars:

1. **GPU-batched parallelism**

528 envs/step $\rightarrow$ 7,920-sample PPO batch on one GPU. The non-stationary ASP objective needs a large batch for stable gradients.

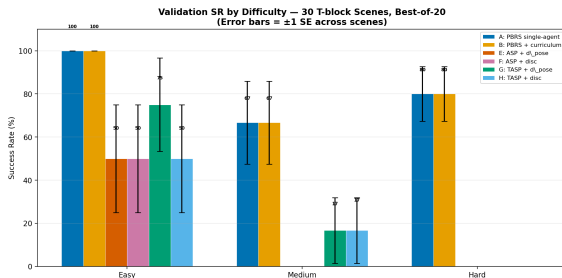
2. **Robotic fidelity**

UR5e + 3D PhysX contact + cuRobo IK = the actual SE(2) task. gym-pusht is a 2D abstraction (diagnostic only).

3. **Time-to-scale (ASP)**

ASP reaches ~ 10 M training pushes in ~ 16 h on GPU vs ~ 5 days on CPU. The

Comparison Summary – All Models with Error Bars



Key numbers (30 T-block scenes):

Model	SR	SE	95% CI
A	80.0%	7.3%	65–95%
B	76.7%	7.7%	61–92%
G	16.7%	6.8%	3–30%
H	10.0%	5.5%	0–21%
E	6.7%	4.6%	0–16%
F	6.7%	4.6%	0–16%

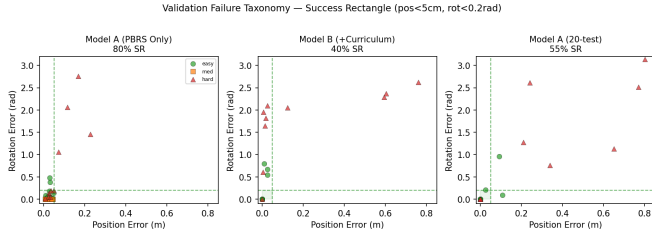
A vs B gap:

3.3pp – **not statistically significant** ($p > 0.05$).

Both A and B solve the task similarly. ASP variants are $4.8\text{--}11.9\times$ below – this gap is significant.

Error bars = ± 1 standard error across scenes. Models C/D excluded – no Isaac 30-scene validation (C: 0% on 20 older, 29

Comparison Summary – Failure Taxonomy



The overarching pattern: structural additions – curriculum staging, second agent, ABC buffer, historical pool – consistently reduce or fail to improve performance. PBRS single-agent (80.0%) is the recommended baseline.



What We Found – Theory vs Practice

ASP did *not* yield a useful curriculum in this setting

In this contact-rich, IK-gated, multi-objective SE(2) pushing task under a single-GPU budget, our ASP implementation produced 6.7–16.7% validation SR vs 80.0% for single-agent. The adversarial mechanism did not generate a learnable progression of goals. This does *not* refute ASP in general – it demonstrates a failure mode under realistic robotics constraints.

GoalEncoder bottleneck provides no measurable benefit

The 8D latent compression did not improve Bob's ability to learn under ASP. However, this result is confounded – Bob's primary failure is the non-stationary goal distribution, not the observation representation.



What We Found – Theory vs Practice

Forced curriculum: functional but not beneficial

Model B's original curriculum trigger (per-push mean PosErr $< 0.08m$) was mis-specified – the threshold was unreachable by construction. A corrected trigger (episodic position-SR ≥ 0.5) produces a competent model (76.7%), but the staged pos $\rightarrow$ rot curriculum does *not* beat the simpler no-curriculum baseline (80.0%). Curriculum works – it just doesn't add value in this setting.

PBRS theory strongly supported (Ng et al. 1999; Grześ 2017)

The policy invariance theorem holds in practice, even in contact-rich manipulation with stochastic PhysX physics. PBRS provides clean, bounded, Markovian rewards. Parameters $k_p=30$, $k_r=5$, $w=10.0$ are robust across all single-agent runs.



Limitations & Research Gaps

What we couldn't test (training budget limited to 3000 iters $\times$ 528 envs):

- **Model I/J:** Bob time penalty $R_B += -\gamma_{sp} \cdot t_B$ (full Sukhbaatar symmetric reward). Could create zero-sum dynamics that stabilize ASP training
- **Model K/L:** time-based ASP + ABC enabled ($\beta_{abc} = 0.5$). ABC might boot after Alice converges
- Larger models (Transformer, diffusion policy), sim-to-real transfer (physical UR5e)

Remaining literature gaps (from systematic survey of 81 papers):

- No paper on SE(2) metric design specifically for RL reward shaping
- No paper on exploiting continuous rotational symmetry (SO(2)) in object manipulation RL
- No formal convergence analysis of effort-based self-play rewards



Future Work – Promising Directions

1. **Zero-sum time-based ASP (Model I):** Bob penalty creates near-zero-sum game. From differentiable game theory (Letcher et al. 2019), zero-sum games have convergent (Hamiltonian) dynamics – this could stabilize ASP where general-sum diverges
2. **ABC after Alice convergence (Model K):** Currently ABC fails because Alice acts randomly ($bc_ratio \approx 1.0$). If Alice converges first, Bob starts with a meaningful imitation target
3. **Relaxed combined success gate:** The combined pos+rot gate is the bottleneck (70% single-agent vs 0–10% for ASP). Partial credit for one objective could provide gradient to learn both
4. **PAIRED (Dennis et al. 2020):** Antagonist minimizes protagonist's regret rather than maximizing difficulty – solvable but challenging environments, addressing the exact failure mode

Conclusion



1. PBRs solves planar pushing (RQ1)

Model A achieves **80.0% validation SR** on 30 held-out T-block scenes – 100% pos-only, 70% pos+rot – with a simple single-agent PPO architecture and no curriculum. PBRs preserves the optimal policy while providing dense, bounded, Markovian gradients. This is the **recommended baseline**.

2. ASP does not improve performance in this domain (RQ2)

All ASP variants (E–H) score 6.7–16.7%, a $4.8\text{--}11.9\times$ gap from single-agent. Time-based Alice (G, 16.7%) partially rescues the adversarial dynamics but cannot close the gap. Complexity adds no benefit.

Conclusion



3. Reward and curriculum are independent (RQ3)

Adding curriculum (B, 76.7%) or self-play (E-H, 6.7–16.7%) does not improve PBRS single-agent performance. They are independent levers – diagnose and optimize separately.

Start simple. PBRS single-agent PPO at 528 envs trains in ~ 1.6 days on an RTX Pro 6000 and achieves 80% SR. Add complexity only when proven to help – and expect it usually won't.



university of
 groningen

Thank You – Questions?

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026



Appendix Outline

- A. Push Mechanics – Limit Surface & Characteristic Length L
- B. Training curves for all PBRS models (Success Rate, Reward, Loss)
- C. PBRS parameter sensitivity (k_p , k_r , w_{pos} , w_{rot})
- D. Validation scene descriptions (30 test cases)
- E. cuRobo IK pipeline and workspace configuration
- F. Observation space specifications (all model variants)
- G. Per-test error bars – A vs B head-to-head and all-model comparison (95% binomial CIs)

Appendix A – Push Mechanics: Limit Surface & L

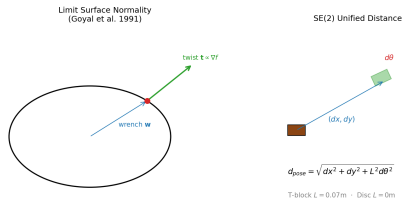


Limit surface normality (Goyal 1991):

$t \propto \nabla f(\mathbf{w})$ – twist is normal to the surface, so every push couples translation *and* rotation.

Characteristic length L (radius of gyration):

- **T-block** $L = 0.07$ m (strong coupling);
disc $L = 0$ m (decoupled)
- L is a physical constant, *not* a hyperparameter

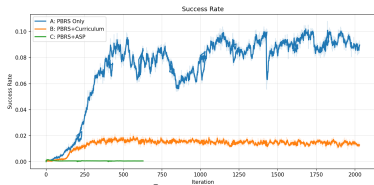


T-block rotates under an off-centre push; disc only slides

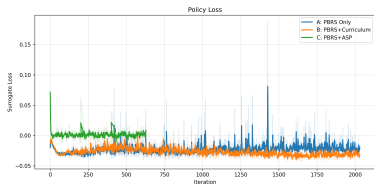
Appendix B – Training Curves (All PBRs Models)



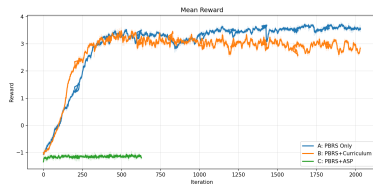
Success Rate



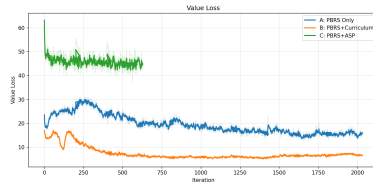
Policy Loss



Mean Reward



Value Loss

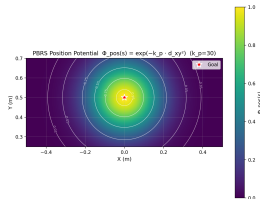


Appendix C – PBRs Theory Analysis



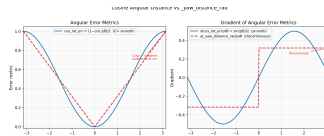
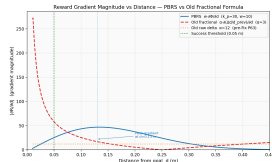
Potential Landscape

$$\Phi(s) = w \cdot \exp(-k \cdot d^2)$$



Gradient Comparison

PBRs vs Fractional Formula



Cosine Angular Distance $c(s) = (1 - \cos(\theta^* - \theta))/2$

Appendix G – Per-Test Error Bars (95% Binomial CIs)

